

Skill4

To Split Input into Tokens, Identify Delimiters, Handle Whitespace, Create Token Structures, Validate Token Streams, Debug Parsing Output.

To Design Parser Logic, Generate Parse Trees, Validate Syntax, Detect Errors, Handle Empty Commands, Produce Execution Structures.

```
#include <stdio.h>
```

```
#include <string.h>
```

```
#include <stdlib.h>
```

```
#define MAX_INPUT 256
```

```
#define MAX_TOKENS 20
```

```
#define MAX_TOKEN_LENGTH 50
```

```
/* Structure for storing tokens */
```

```
typedef struct
```

```
{
```

```
    char value[MAX_TOKEN_LENGTH];
```

```
} Token;
```

```
/* Tokenize input */
```

```
int tokenize(char *input, Token tokens[])
```

```
{
```

```
    int count = 0;
```

```
    char *token;
```

```

/* Split using delimiters */
token = strtok(input, " \t\n");

while (token != NULL && count < MAX_TOKENS)
{
    strncpy(
        tokens[count].value,
        token,
        MAX_TOKEN_LENGTH - 1
    );

    tokens[count].value[MAX_TOKEN_LENGTH - 1] = '\0';

    count++;

    token = strtok(NULL, " \t\n");
}

return count;
}

/* Validate token stream */
int validate_tokens(Token tokens[], int count)
{
    if (count == 0)
    {

```

```

    printf("Error: Empty command.\n");

    return 0;
}

for (int i = 0; i < count; i++)
{
    if (strlen(tokens[i].value) == 0)
    {
        printf("Error: Invalid token.\n");

        return 0;
    }
}

return 1;
}

```

```

/* Debug output */
void print_tokens(Token tokens[], int count)
{
    printf("\n----- TOKEN STREAM ----- \n");

    for (int i = 0; i < count; i++)
    {
        printf(
            "Token %d: %s\n",

```

```

        i,
        tokens[i].value
    );
}

printf("-----\n");
}

int main()
{
    char input[MAX_INPUT];
    Token tokens[MAX_TOKENS];
    printf("TOKENIZER PROGRAM\n");
    printf("Enter command: ");
    fgets(
        input,
        sizeof(input),
        stdin
    );
    /* Tokenize input */
    int token_count =
        tokenize(input, tokens);
    /* Validate tokens */
    if (!validate_tokens(
        tokens,
        token_count
    ))

```

```

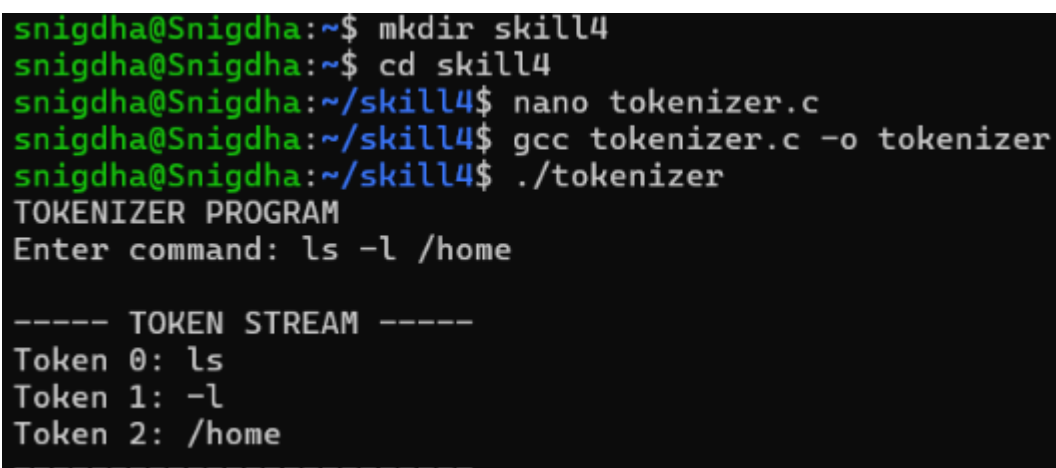
{
    return 1;
}

/* Debug token output */

print_tokens(
    tokens,
    token_count
);

return 0;
}

```



```

snigdha@Snigdha:~$ mkdir skill4
snigdha@Snigdha:~$ cd skill4
snigdha@Snigdha:~/skill4$ nano tokenizer.c
snigdha@Snigdha:~/skill4$ gcc tokenizer.c -o tokenizer
snigdha@Snigdha:~/skill4$ ./tokenizer
TOKENIZER PROGRAM
Enter command: ls -l /home

----- TOKEN STREAM -----
Token 0: ls
Token 1: -l
Token 2: /home
=====

```

```
#include <stdio.h>
```

```
#include <string.h>
```

```
#include <stdlib.h>
```

```
#define MAX_INPUT 256
```

```
#define MAX_TOKENS 20
```

```
#define MAX_TOKEN_LENGTH 50
```

```
/* Token structure */  
  
typedef struct  
{  
    char value[MAX_TOKEN_LENGTH];  
  
} Token;
```

```
/* Execution structure */  
  
typedef struct  
{  
    char command[MAX_TOKEN_LENGTH];  
  
    char arguments[MAX_TOKENS][MAX_TOKEN_LENGTH];  
  
    int argument_count;  
  
} Command;
```

```
/* Tokenize input */  
  
int tokenize(char *input, Token tokens[])  
{  
    int count = 0;  
  
    char *token;
```

```
token = strtok(  
    input,  
    "\\t\\n"  
);
```

```
while (  
    token != NULL &&  
    count < MAX_TOKENS  
)  
{  
    strncpy(  
        tokens[count].value,  
        token,  
        MAX_TOKEN_LENGTH - 1  
    );
```

```
tokens[count]  
    .value[MAX_TOKEN_LENGTH - 1] =  
    '\\0';
```

```
count++;
```

```
token = strtok(  
    NULL,  
    "\\t\\n"  
);
```

```

    }

    return count;
}

/* Validate token stream */
int validate_tokens(
    Token tokens[],
    int count
)
{
    if (count == 0)
    {
        printf(
            "Syntax Error: Empty command.\n"
        );

        return 0;
    }

    /* First token must exist */
    if (
        strlen(
            tokens[0].value
        ) == 0
    )

```

```
{  
    printf(  
        "Syntax Error: Missing command.\n"  
    );  
  
    return 0;  
}
```

```
    return 1;  
}
```

```
/* Parser */  
int parse_command(  
    Token tokens[],  
    int count,  
    Command *cmd  
)  
{  
    if (count == 0)  
    {  
        return 0;  
    }
```

```
/* First token is command */  
strcpy(  

```

```
cmd->command,  
tokens[0].value  
);
```

```
cmd->argument_count = 0;
```

```
/* Remaining tokens are arguments */
```

```
for (  
    int i = 1;  
    i < count;  
    i++)  
)  
{  
    strcpy(  
        cmd->arguments[  
            cmd->argument_count  
        ],  
        tokens[i].value  
    );
```

```
    cmd->argument_count++;  
}
```

```
return 1;
```

```
}
```

```
/* Display parse tree */
```

```
void print_parse_tree(
```

```
    Command *cmd
```

```
)
```

```
{
```

```
    printf(
```

```
        "\n----- PARSE TREE ----- \n"
```

```
);
```

```
    printf(
```

```
        "COMMAND \n"
```

```
);
```

```
    printf(
```

```
        " | \n"
```

```
);
```

```
    printf(
```

```
        " |-- %s \n",
```

```
        cmd->command
```

```
);
```

```

for (
    int i = 0;
    i < cmd->argument_count;
    i++
)
{
    printf(
        " |-- %s\n",
        cmd->arguments[i]
    );
}

printf(
    "-----\n"
);
}

/* Display execution structure */
void print_execution_structure(
    Command *cmd
)
{
    printf(
        "\nEXECUTION STRUCTURE\n"
    );
}

```

```
printf(  
    "Command: %s\n",  
    cmd->command  
);
```

```
printf(  
    "Number of Arguments: %d\n",  
    cmd->argument_count  
);
```

```
for (  
    int i = 0;  
    i < cmd->argument_count;  
    i++)  
{  
    printf(  
        "Argument %d: %s\n",  
        i + 1,  
        cmd->arguments[i]  
    );  
}  
}
```

```
int main()
{
    char input[MAX_INPUT];

    Token tokens[MAX_TOKENS];

    Command cmd;

    printf(
        "SIMPLE COMMAND PARSER\n"
    );
    printf(
        "Enter command: "
    );
    fgets(
        input,
        sizeof(input),
        stdin
    );
    /* TOKENIZATION */
    int token_count =
        tokenize(
            input,
            tokens
        );
    /* VALIDATION */
    if (
        !validate_tokens(
```

```

        tokens,
        token_count
    )
)
{
    return 1;
}
/* PARSING */
if (
    !parse_command(
        tokens,
        token_count,
        &cmd
    )
)
{
    printf(
        "Parsing failed.\n"
    );
    return 1;
}
/* PRINT PARSE TREE */
print_parse_tree(
    &cmd
);
/* PRINT EXECUTION STRUCTURE */
print_execution_structure(
    &cmd

```

```
);  
return 0;  
}
```

```
-----  
snigdha@Snigdha:~/skill4$ nano parser.c  
snigdha@Snigdha:~/skill4$ gcc parser.c -o parser  
snigdha@Snigdha:~/skill4$ ./parser  
SIMPLE COMMAND PARSER  
Enter command: ls -l /home  
  
----- PARSE TREE -----  
COMMAND  
|  
|-- ls  
|-- -l  
|-- /home  
-----  
  
EXECUTION STRUCTURE  
Command: ls  
Number of Arguments: 2  
Argument 1: -l  
Argument 2: /home  
snigdha@Snigdha:~/skill4$ ./parser  
SIMPLE COMMAND PARSER  
Enter command: pwd  
  
----- PARSE TREE -----  
COMMAND  
|  
|-- pwd  
-----  
  
EXECUTION STRUCTURE  
Command: pwd  
Number of Arguments: 0  
snigdha@Snigdha:~/skill4$
```